

Universidad Técnica Federico Santa María
Sistemas Distribuidos
Segundo Semestre 2026

Laboratorio 3: Sistemas Distribuidos

Operación Gamma

Profesor: Jorge Díaz
Ayudantes de Lab: Felipe Marchant V. & Nelson Sepúlveda V.

Octubre 2026

1 Objetivos del Laboratorio

- Comprender e implementar un Servicio de Nombres (Naming) que permita a las entidades de un sistema distribuido localizarse dinámicamente, sin conocer de antemano la ubicación física de las demás.
- Comprender e implementar mecanismos de Coordinación mediante el algoritmo de elección de líder Bully.
- Distribuir el procesamiento de datos entre múltiples réplicas de una entidad, coordinadas por un líder que reparte el trabajo.
- Desplegar el ecosistema en un entorno distribuido real compuesto por 4 Máquinas Virtuales, integrando la ingesta de cambios en una base de datos MySQL dockerizada.

2 Tecnologías

- El lenguaje de programación a utilizar es **Go**.
- Para la comunicación asíncrona de ingesta se mantendrá el gestor de colas **RabbitMQ**.
- Para la comunicación síncrona interna se mantendrá **gRPC** y **Protocol Buffers (protobuf)**.
- Para el catálogo se utilizará una base de datos **MySQL** dockerizada.
- Para el despliegue de los servicios se seguirá utilizando **Docker** y **Docker Compose**, ahora sobre las Máquinas Virtuales asignadas a cada grupo.

3 Laboratorio

3.1 Contexto

En los últimos meses, en la ciudad de USM, una unidad policial ha detectado un aumento sostenido de actividades asociadas a una organización dedicada al narcotráfico. La organización, identificada durante la investigación como **Los Distros**, opera mediante distintos grupos que realizan actividades en diferentes zonas de la ciudad y presentan distintos niveles de peligrosidad. Hasta el momento, la policía posee información limitada sobre su estructura interna, sus integrantes y las actividades que realizan.

A partir de estos antecedentes se inicia la **Operación Gamma**, cuyo objetivo es identificar las actividades realizadas por Los Distros y recopilar información sobre sus integrantes. Un **Agente Secreto** infiltrado obtiene esas actividades y las transmite sin modificarlas. El **Cuartel Policial** construye el registro de la investigación y el **Policía** lo consulta desde una terminal.

La operación ya cuenta con un flujo simple: Los Distros crean la actividad, el Agente la reenvía, el Cuartel la recuerda y el Policía pide la lista. Ese flujo alcanza mientras todas las entidades viven en la misma red y una sola copia hace el trabajo. Al repartir el sistema en varias máquinas, esa red deja de resolver los nombres por sí sola, y una sola copia del trabajo se vuelve un punto único de falla. Además, el registro debe sobrevivir en una base de datos que empieza vacía y se llena con cada actividad que llega al Cuartel. En este laboratorio Los Distros ya no generan actividades: leen las filas de un CSV y publican cada una.

3.2 Explicación

3.2.1 Flujo anterior

El flujo anterior usa cuatro entidades y no tiene base de datos, servidor de nombres ni réplicas.

Los Distros crean una actividad. Cada actividad tiene identificador, ubicación, nivel de peligrosidad e integrantes. La publican en RabbitMQ y siguen trabajando sin esperar respuesta.

El Agente Secreto consume ese mensaje. No cambia ningún campo. Lo reenvía hacia el Cuartel y continúa.

El Cuartel recibe la actividad y la guarda en memoria, en una colección interna del programa.

El Policía pide la lista mediante una llamada gRPC. El Cuartel lee la memoria y responde. El Policía imprime las actividades y los integrantes. Esa impresión es la última acción del flujo anterior.

Con dos actividades, el recorrido es el siguiente:

1. Los Distros crean `A1` (Puerto, Alta, Nico y Vale) y se la envían al Agente.
2. El Agente reenvía `A1` sin modificarla.
3. El Cuartel guarda `A1` en memoria.
4. Los Distros crean `A2` (Centro, Media, Nico) y se repite el mismo camino.

5. El Policía pide la lista. El Cuartel responde desde memoria y el Policía imprime:

```
A1 | Puerto | Alta  | Nico, Vale
A2 | Centro | Media | Nico
```

Si el Cuartel se detiene, la memoria se pierde. La dirección de cada entidad está fija en la configuración, y una sola instancia procesa todo el trabajo.

3.2.2 Flujo nuevo

El flujo nuevo conserva el recorrido hasta el Policía: el Agente no modifica la actividad y el Policía termina pidiendo la lista. Los Distros ya no generan actividades. Leen un CSV y publican sus filas. Cambian además tres cosas. La actividad queda guardada en MySQL, las direcciones gRPC se obtienen del Servidor de Nombres, y tres procesadoras eligen un líder que reparte el trabajo.

La base empieza vacía. Solo existe la tabla de actividades. Nadie la carga de antemano. La primera fila aparece cuando el Cuartel recibe la primera actividad.

El recorrido de `A1` es el siguiente:

1. **Los Distros** leen `A1` (Puerto, Alta, Nico y Vale) desde el CSV y la publican en la cola del Agente.
2. **El Agente Secreto** la recibe y no modifica ningún campo. Pregunta al Servidor de Nombres dónde está `gamma.procesamiento.lider` y le entrega `A1` a esa dirección.
3. **El líder** reparte `A1`. Se la queda una procesadora.
4. **Esa procesadora** pregunta dónde está `gamma.almacenamiento.cuartel` y le entrega `A1` al Cuartel.
5. **El Cuartel** hace `INSERT` de `A1`. La base pasa de cero filas a una.
6. `A2` es la fila siguiente del CSV (Centro, Media, Nico) y recorre los mismos cinco pasos. La base queda con dos filas.
7. **El Policía** pregunta dónde está `gamma.almacenamiento.cuartel`, pide la lista y espera la respuesta. El Cuartel hace `SELECT` y el Policía imprime:

```
A1 | Puerto | Alta  | Nico, Vale
A2 | Centro | Media | Nico
```

Esa impresión sigue siendo la última acción. Los Distros leen las filas de una de las tres opciones en `data/` y publican cada una, en ese orden, por el mismo flujo. En `opcion-1.csv` la tercera fila repite `A1`. El Cuartel reconoce el identificador y no inserta otra fila, así la lista no muestra duplicados.

Naming. El Servidor de Nombres es la guía del sistema. Guarda cada nombre junto con una dirección `IP:puerto` y una hora de vencimiento. Ofrece cuatro operaciones: `Registrar`, `Renovar`, `Resolver` y `Listar`. Cada registro dura unos 10 segundos y la entidad lo renueva cada 3 segundos. Si nadie renueva, el nombre desaparece, incluido el alias del líder. La única dirección escrita en la configuración es la del Servidor de Nombres. Toda llamada gRPC parte de un `Resolver`.

Los nombres del sistema son:

```
gamma.procesamiento.procesador1
gamma.procesamiento.procesador2
gamma.procesamiento.procesador3
gamma.procesamiento.lider
gamma.almacenamiento.cuartel
```

Coordinación. Las tres procesadoras ejecutan el mismo programa. El identificador 1, 2 o 3 llega por variable de entorno. Al iniciar se registran y usan `Listar("gamma.procesamiento.")` para conocer a sus pares. Luego eligen líder con el algoritmo Bully:

1. Quien detecta que no hay líder envía `ELECCION` a las procesadoras de identificador mayor.
2. Si ninguna responde a tiempo, se proclama líder, envía `COORDINADOR` al resto y registra `gamma.procesamiento.lider` con su propia dirección.
3. Si una de identificador mayor responde `OK`, espera `COORDINADOR`. Si ese aviso no llega, vuelve a elegir.
4. Quien recibe `ELECCION` de un identificador menor responde `OK` e inicia su propia elección.

Con las tres activas, la procesadora 3 queda como líder. Las demás le hacen un `Ping` cada 3 segundos. Si el líder deja de responder, las restantes eligen de nuevo y la ganadora actualiza el alias. El Agente, al fallar la entrega, olvida la dirección que tenía, vuelve a resolver `gamma.procesamiento.lider` y reenvía la misma actividad.

3.3 Entidades

3.3.1 Los Distros

Los Distros corresponden a la organización que origina las actividades de la Operación Gamma. Su misión es publicar esas actividades hacia el Agente Secreto para que la investigación pueda registrarlas. Se despliegan en la **MV1**, junto a la base de datos.

En el flujo anterior generaban cada actividad durante la ejecución. En el flujo nuevo ya no las generan. Leen un CSV y publican las filas de ese archivo. Cada fila describe un hecho distinto, con su propia ubicación, nivel de peligrosidad e integrantes. Publican y continúan: no esperan a que el Agente, el Cuartel ni el Policía terminen su trabajo.

Funciones de Los Distros:

- **Lectura del CSV:** Leen una sola opción: `data/opcion-1.csv`, `data/opcion-2.csv` o `data/opcion-3.csv`. La opción llega por variable de entorno. Recorren las filas de arriba a abajo y no se saltan ninguna.
- **Publicación asíncrona:** Cada fila se publica en la cola de RabbitMQ del Agente Secreto. La cola vive en la **MV2**. Los Distros no esperan a que el Agente, el líder ni el Cuartel terminen.
- **Identificador único:** Cada fila trae `actividad_id`. Si el archivo repite ese valor, Los Distros lo publican otra vez. El Cuartel reconoce el reenvío y no guarda la misma fila dos veces.
- **Marca de tiempo:** Usan la `fecha_emision` de la fila. No la reemplazan con `time.Now()`.

Reglas de lectura

Los Distros no generan ni inventan actividades durante la ejecución. Solo leen el CSV. La base empieza vacía y ellos no leen ni escriben en ella. El separador del archivo es `;` y las columnas son `actividad_id`, `ubicacion`, `peligrosidad`, `integrantes` y `fecha_emision`. Los integrantes vienen separados por coma en esa columna y se publican como arreglo JSON.

Cada fila debe incluir un identificador, una ubicación, una peligrosidad, al menos un integrante y la fecha de emisión. Publicar una fila recorre el flujo completo: el Agente resuelve al líder, una procesadora entrega la actividad al Cuartel y el Cuartel la inserta en MySQL.

Estructura del mensaje de actividad (ejemplo):

Listing 1: Ejemplo de actividad de Los Distros

```
{
  "actividad_id": "A1",
  "ubicacion": "Puerto",
  "peligrosidad": "Alta",
  "integrantes": ["Nico", "Vale"],
  "fecha_emision": "2026-10-15T22:10:00"
}
```

3.3.2 Agente Secreto

El Agente Secreto es el infiltrado que recoge lo que publican Los Distros. Su misión es recibir cada actividad y transmitirla, sin modificarla, hacia quien corresponda para que llegue al Cuartel. Se despliega en la **MV2**, junto a RabbitMQ y la Procesadora 1.

En el flujo anterior reenviaba la actividad directo al Cuartel. En el flujo nuevo la entrega al líder de las procesadoras. Para saber dónde está ese líder, consulta al Servidor de Nombres.

Funciones del Agente Secreto:

- **Consumo de la cola:** Lee las actividades publicadas por Los Distros en RabbitMQ.
- **Reenvío intacto:** Mantiene ubicación, peligrosidad, integrantes, identificador y fecha tal como llegaron.
- **Resolución del líder:** Pide al Servidor de Nombres la dirección de `gamma.procesamiento.lider` y envía la actividad por gRPC a esa dirección.
- **Reintento:** Si la entrega falla, olvida la dirección guardada, vuelve a resolver el alias y reenvía la misma actividad, con el mismo `actividad_id`.

Regla de tránsito

El Agente no inserta en la base, no arma la lista y no decide cuál procesadora es la líder. Solo mueve el mensaje hasta el líder vigente.

3.3.3 Servidor de Nombres

El Servidor de Nombres es la guía del sistema. Su misión es asociar un nombre lógico con la dirección física de quien lo usa, para que las entidades se encuentren entre máquinas virtuales. Se despliega en la **MV3**, junto a la Procesadora 2.

Es la única dirección que las demás entidades conocen de antemano, mediante variable de entorno. Cualquier otra llamada gRPC parte de una consulta a esta guía.

Funciones del Servidor de Nombres:

- **Registrar:** Guarda un nombre junto con su `IP:puerto`. El registro vence a los 10 segundos.
- **Renovar:** Alarga el vencimiento. Cada entidad registrada lo llama cada 3 segundos.
- **Resolver:** Dado un nombre, devuelve la dirección vigente. Si el nombre no existe o ya venció, responde error.
- **Listar:** Devuelve los nombres activos bajo un prefijo. Las procesadoras lo usan con `gamma.procesamiento.` para conocer a sus pares.
- **Expiración:** Borra solo los registros que no fueron renovados a tiempo, incluido el alias del líder.

Nombres del sistema

```
gamma.procesamiento.procesador1
gamma.procesamiento.procesador2
gamma.procesamiento.procesador3
gamma.procesamiento.lider
gamma.almacenamiento.cuartel
```

La dirección que se registra es la IP de la máquina virtual y el puerto publicado, no la IP interna del contenedor.

3.3.4 Procesadoras

Las Procesadoras corresponden a tres réplicas del mismo programa, con identificadores **1, 2 y 3**. Su misión es elegir un líder y repartir cada actividad que trae el Agente, hasta entregarla al Cuartel. La Procesadora 1 corre en la **MV2**, la Procesadora 2 en la **MV3** y la Procesadora 3 en la **MV4**.

Las tres ejecutan el mismo código. El identificador llega por variable de entorno. El rol de líder o trabajadora sale de la elección, no de la configuración. Con las tres activas, la de identificador mayor queda como líder: la Procesadora 3.

Funciones de las Procesadoras:

- **Registro:** Cada una se registra como `gamma.procesamiento.procesadorN` y renueva ese registro.
- **Descubrimiento:** Consultan `Listar("gamma.procesamiento.")` para conocer a las pares vivas.

- **Elección de líder:** Aplican el algoritmo Bully. Quien gana registra el alias `gamma.procesamiento.lider` con su propia dirección.
- **Reparto:** El líder recibe la actividad del Agente y la asigna a una procesadora activa, incluida ella misma.
- **Entrega al Cuartel:** La procesadora que queda con la actividad resuelve `gamma.almacenamiento.cuartel` y se la entrega por gRPC.
- **Vigilancia:** Las trabajadoras hacen `Ping` al líder cada 3 segundos. Si no responde, se inicia una nueva elección y el alias pasa a la nueva líder.

Reglas de la elección

Una procesadora que detecta la ausencia de líder envía `ELECCION` a las de identificador mayor. Si ninguna responde a tiempo, se proclama líder y envía `COORDINADOR` al resto. Si una mayor responde `OK`, espera el `COORDINADOR`; si ese aviso no llega, vuelve a elegir. Quien recibe `ELECCION` de un identificador menor responde `OK` e inicia su propia elección.

La procesadora no modifica la actividad ni escribe en la base. Esa escritura pertenece al Cuartel.

3.3.5 Cuartel Policial

El Cuartel Policial es el registro de la investigación. Su misión es recibir cada actividad ya repartida, guardarla en la base y responder cuando el Policía pide la lista. Se despliega en la **MV4**, junto a la Policía y la Procesadora 3.

En el flujo anterior guardaba las actividades en memoria. En el flujo nuevo la memoria deja de ser el registro: cada actividad se inserta en MySQL y la lista se lee desde ahí.

Funciones del Cuartel:

- **Registro en la guía:** Al iniciar se registra como `gamma.almacenamiento.cuartel` y renueva ese nombre.
- **Recepción:** Atiende por gRPC la actividad que le entrega una procesadora.
- **Escritura:** Hace `INSERT` de la actividad. La base, que partió vacía, gana una fila con cada actividad nueva. `integrantes` se guarda como un arreglo JSON y `fecha_emision` como `DATETIME`.
- **Idempotencia:** Si llega de nuevo el mismo `actividad_id`, la clave primaria ya existe. El `INSERT` no agrega otra fila y la actividad guardada se mantiene.
- **Consulta:** Cuando el Policía pide la lista, hace `SELECT` y devuelve las filas guardadas.

Regla de acceso

El Cuartel es la única entidad que escribe y lee la base. Los Distros, el Agente, las procesadoras y el Policía no abren una conexión hacia MySQL.

3.3.6 Base de datos MySQL

La base de datos es el registro persistente de la Operación Gamma. Su misión es guardar las actividades que el Cuartel va recibiendo y devolverlas cuando el Cuartel pide la lista. Corre en Docker en la **MV1**, junto a Los Distros.

Usa la imagen oficial `mysql:8`. No hay una imagen propia con el esquema adentro. El esquema es el mismo para todos los grupos y está en `init.sql`. Docker lo ejecuta una sola vez, al crear el volumen, desde `/docker-entrypoint-initdb.d/`. El script crea la tabla en InnoDB, con `utf8mb4`, y no inserta filas. La primera fila aparece con el primer `INSERT` del Cuartel.

Estructura inicial

Listing 2: Esquema inicial de la Operación Gamma

```
CREATE TABLE actividades (
    actividad_id  VARCHAR(64)  NOT NULL,
    ubicacion     VARCHAR(255) NOT NULL,
    peligrosidad  VARCHAR(32)  NOT NULL,
    integrantes   JSON         NOT NULL,
    fecha_emision DATETIME     NOT NULL,
    PRIMARY KEY (actividad_id),
    CONSTRAINT chk_actividades_integrantes CHECK (
        JSON_TYPE(integrantes) = 'ARRAY' AND JSON_LENGTH(integrantes) >= 1
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

`actividad_id` es la clave primaria. Un reenvío de la misma actividad no crea otra fila. `integrantes` es un arreglo JSON en esa misma fila, por ejemplo `["Nico", "Vale"]`. `fecha_emision` se guarda como `DATETIME`, sin zona horaria.

Funciones de la base de datos:

- **Tabla vacía al partir:** `init.sql` crea `actividades` al levantar el contenedor, sin datos de prueba.
- **Alta de actividades:** Cada `INSERT` del Cuartel agrega una actividad con su identificador, ubicación, peligrosidad, integrantes y fecha de emisión.
- **Lectura de la lista:** El `SELECT` del Cuartel devuelve las filas acumuladas para que el Policía las imprima.
- **Una fila por identificador:** El identificador de la actividad es único, así un reenvío no duplica el registro.

3.3.7 Policía

El Policía es quien consulta el estado de la investigación. Su misión es pedir la lista de actividades y mostrarla en la terminal. Se despliega en la **MV4**, junto al Cuartel.

Esta consulta es la última acción del flujo. El Policía espera la respuesta antes de imprimir.

Funciones del Policía:

- **Resolución del Cuartel:** Pregunta al Servidor de Nombres dónde está `gamma.almacenamiento.cuartel`.
- **Solicitud de la lista:** Llama por gRPC al Cuartel y espera el resultado del `SELECT`.
- **Impresión:** Muestra en la terminal las actividades y los integrantes conocidos hasta ese momento.

Con `A1` y `A2` ya insertadas, la salida es:

```
A1 | Puerto | Alta  | Nico, Vale
A2 | Centro | Media | Nico
```

4 Uso de Docker y Máquinas Virtuales

Todos los procesos del laboratorio deben ejecutarse en contenedores Docker independientes dentro de las 4 máquinas virtuales asignadas a cada grupo. Para facilitar la evaluación de la simulación de fallos, se deberá respetar la siguiente distribución:

Máquina virtual	**Entidades**
MV1	MySQL, Los Distros
MV2	RabbitMQ, Agente Secreto, Procesadora 1
MV3	Servidor de Nombres, Procesadora 2
MV4	Procesadora 3, Cuartel, Policía

La red interna de Docker Compose no se extiende entre máquinas virtuales. Las entidades deben publicar sus puertos en la máquina virtual y registrarse en el Servidor de Nombres con la IP de esa máquina y el puerto publicado, no con la IP interna del contenedor.

5 Restricciones

Las librerías de Golang permitidas son:

- `time`, `strconv`, `strings`, `math`, `math/rand`, `net`, `context`, `fmt`, `log`, `os`, `os/signal`, `sync`, `bufio`, `errors`, `io`, `sort`, `encoding/csv`, `encoding/json`, `flag`
- `amqp` (para RabbitMQ)
- `grpc` y `protobuf` (para la comunicación síncrona)
- `database/sql` junto con el driver `github.com/go-sql-driver/mysql` (para MySQL)

Todo uso de librerías externas que no se han mencionado en el enunciado debe ser consultado con los ayudantes.

6 Consideraciones

- **Comunicación y flujo de datos:**
  - La publicación de Los Distros hacia el Agente Secreto debe realizarse estrictamente mediante RabbitMQ.
  - Toda la comunicación interna entre el Agente Secreto, las Procesadoras, el Cuartel, el Policía y el Servidor de Nombres debe realizarse estrictamente mediante gRPC.
  - Está prohibido el paso de archivos y el uso de volúmenes compartidos entre entidades. El CSV lo lee únicamente Los Distros, dentro de su propio contenedor.
  - Solo el Cuartel abre conexión hacia MySQL. Los Distros, el Agente, las Procesadoras y el Policía no leen ni escriben en la base.
- **Naming:**
  - Está prohibido escribir en el código o en la configuración la dirección de cualquier entidad que no sea el Servidor de Nombres. Toda conexión gRPC debe partir de un `Resolver`.

  - No se permite reemplazar el Servidor de Nombres por herramientas externas de descubrimiento de servicios, como Consul, etcd o CoreDNS.

- **Coordinación:**
  - La elección de líder debe implementarse con el algoritmo Bully, programado por el grupo.
  - No se permite utilizar herramientas o librerías de coordinación o consenso, como ZooKeeper, etcd o una implementación de Raft.

- **Archivos Protocol Buffers (.proto):**
  - Deberán definir los servicios del Servidor de Nombres, los mensajes de elección (`ELECCION`, `OK` y `COORDINADOR`), el `Ping`, la entrega de una actividad y la consulta de la lista.
  - Los archivos `.proto`, así como el código generado (`.pb.go` y `_grpc.pb.go`), deben estar incluidos en el repositorio y en la entrega.

- **Conexión y tolerancia:**
  - Todas las entidades deben reintentar sus conexiones iniciales a RabbitMQ, MySQL, el Servidor de Nombres y los servidores gRPC. Las máquinas virtuales pueden levantarse en cualquier orden.
  - Ninguna actividad puede perderse ni quedar duplicada en MySQL ante la caída de una o dos Procesadoras.
  - Se asume que el Servidor de Nombres, RabbitMQ, MySQL, Los Distros, el Agente Secreto, el Cuartel y el Policía no fallan, y que la red no se parte. Las únicas fallas que se simularán son las de las Procesadoras.

- **Simulación y entorno:**
  - Las entidades deben generar logs claros que permitan seguir cada fase de la ejecución. Como mínimo se deben observar los registros y las expiraciones en el Servidor de Nombres, cada mensaje del algoritmo Bully, el líder electo, la asignación de cada actividad, los reenvíos del Agente y los duplicados descartados por el Cuartel.
  - Se recomienda el uso de etiquetas que incluyan el rol actual, por ejemplo `[LOS DISTROS]`, `[AGENTE]`, `[NAMESERVER]`, `[PROCESADORA-3][LIDER]`, `[PROCESADORA-1][TRABAJADORA]`, `[CUARTEL]` y `[POLICIA]`.

- **Archivos de configuración:**
  - En la evaluación se podrá usar otra opción de CSV y otros valores de conexión. Nada de eso debe estar escrito en el código. Utilicen variables de entorno, por ejemplo un archivo `.env` por máquina virtual, para la dirección del Servidor de Nombres, las credenciales de MySQL, el nombre de la cola, la opción de CSV, los intervalos y los timeouts.

- **Contenedores Docker y orquestación:**
  - Cada entidad debe ejecutarse en un contenedor Docker independiente, orquestado mediante un `docker-compose.yml` por cada máquina virtual.
  - Se debe incluir un `Makefile` en cada máquina virtual con, al menos, las reglas `make docker-VMX` para levantar las entidades de esa máquina y `make down-VMX` para detenerlas, con X entre 1 y 4.

- **Consultas y soporte:**
  - Se realizará una ayudantía para detallar el enunciado, el uso de las máquinas virtuales y la base de datos.
  - Las consultas deben realizarse en el foro habilitado en Aula o al correo de los ayudantes:
    * `felipe.marchantv@usm.cl`
    * `nelson.sepulvedav@usm.cl`
  - No se responderán consultas enviadas con menos de 48 horas antes de la fecha de entrega original. En caso de extensión de plazo, para esta regla se considerará la fecha original de entrega.

7 Reglas de Entrega

- El laboratorio se entrega en los grupos de 2 a 3 personas previamente asignados en Aula.
- La fecha de entrega es el día 10 de noviembre a las 23:59 hrs.
- La tarea se revisará en las máquinas virtuales, por lo que los archivos necesarios para su ejecución deben estar en ellas. El código debe estar indentado, comentado, sin warnings y sin errores.
- Además de los códigos en las máquinas virtuales y en GitHub, deberán subir un archivo comprimido con todos los códigos, en carpetas separadas por entidad, en formato `.zip` con el nombre `GrupoXX-Lab3.zip`, donde XX es el número del grupo.
- Debe dejar un `Makefile` en cada máquina virtual asignada al grupo para la ejecución de las entidades correspondientes.
- Debe dejar un `README` con el nombre y rol de cada integrante, las instrucciones de ejecución y la distribución de las entidades en las máquinas virtuales.
- El uso de Docker es obligatorio. Las implementaciones que no hagan uso de Docker serán evaluadas con nota 0.
- Las faltas de README y las copias serán evaluadas con nota 0.
- Todos los descuentos y aclaraciones están en el documento "Reglas de Laboratorio", disponible en el Aula virtual.